

Basic (your first working tool) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper LangChain/Python). Read Basic first even if you already know this — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

You need a normal Python function, marked as a "tool" the model is allowed to use.

The `@tool` decorator does that marking. Under it, write a completely ordinary function — it does the actual math, nothing fancy.

Then, when you call the model, you tell it "here's a list of tools you're allowed to use" — and ask a question that clearly needs one of them.

The model won't compute the answer itself here — it will ask to *call* your function. Your job in this exercise is to see that request happen, print it, then actually run the function yourself and print the real result.

Things to use:

- `from langchain_core.tools import tool`
- `@tool` above your function, with a docstring describing what it does.
- `model_with_tools = ChatOpenAI().bind_tools([add])`
- `response = model_with_tools.invoke("What's 5 + 7?")`
- `print(response.tool_calls)`

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

`@tool` (from `langchain_core.tools`) wraps a normal typed function into a `Tool` object that carries its name, its docstring (used as the description), and its argument shape — all derived automatically from the function's own signature and type hints. This is why type hints on tool functions aren't optional the way they might be on a throwaway script: the model literally reads them.

The docstring matters just as much — `@tool` reads it as the tool's description shown to the model, so a missing or vague docstring is exactly the "vague description" problem the Core Concepts section warns about, even on this first, simple example.

Registering the tool on a model call — `model.bind_tools([add])` — returns a new model wrapped with that tool list attached. `bind_tools()` takes a list even for one tool, since it's the same shape you'll extend in later exercises with more tools. Calling `.invoke(...)` on it doesn't run your function; it returns an `AIMessage` whose `.tool_calls` attribute is a list of dictionaries, each with `name`, `args`, and an `id` — that's everything your code needs to actually go run the right function with the right arguments.

Actually running it: `add.invoke(response.tool_calls[0]["args"])` — note this uses `.invoke(...)` on the tool itself, not a plain Python function call, since `@tool` wraps it into a `Runnable` too.

Write out, in plain words, the two-step handoff: the model requests a call, and your code is the one that actually executes it.

Worth knowing beyond this exercise: "run the tool and print the real result" isn't actually the end of a real tool-calling round trip. The Core Concepts section's 5-step trace has 2 more steps after this — sending the result back to the model, tagged with `call["id"]`, via `ToolMessage`, and a second `.invoke(...)` call so the model produces its actual final answer. Not needed here; Doc07's agent loop is exactly that pattern, run repeatedly.

Difference between Basic and Intermediate: Basic names the pieces and stops once the tool has actually run once. Intermediate explains what each of those pieces does under the hood, and adds real type-hint/docstring awareness.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
define a tool called add that takes two numbers and returns their sum

register it on a model

ask the model "what's 5 + 7?"

print what tool call the model requested (name, arguments)

actually run the tool with those arguments

print the real result
```

Here's almost the whole thing — just try running it and reading it line by line:

```
# first_tool_call_practice.py
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

@tool
def add(a: int, b: int) -> int:
    """Add two integers together and return the sum."""
    return a + b

model_with_tools = ChatOpenAI().bind_tools([add])
```

Expected output if you run just this (nothing calls the model yet): nothing — building `model_with_tools` doesn't ask it anything. Add `response = model_with_tools.invoke("What's 5 + 7?")` and `print(response.tool_calls)` below it to see the request print.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
@tool
def add(a: int, b: int) -> int:
    """Add two integers together and return the sum."""
    return a + b

model_with_tools = ChatOpenAI(model="...", temperature=0).bind_tools([add])

response = model_with_tools.invoke("What's 5 + 7?")
print(response.tool_calls)

for call in response.tool_calls:
    if call["name"] == "add":
        result = add.invoke(call["args"])
        print(f"Real result: {result}")
```

What's missing from a real ship-it version: checking `response.tool_calls` isn't empty before looping (the model could just answer directly), and checking `call["name"]` matches what you expect instead of assuming it. Add both, then compare against the [Solution](#).

Difference between Basic and Intermediate: same underlying idea (register, ask, run) at 2 completeness levels — Basic's version stops the instant it has the real result of one call, Intermediate adds the defensive checks a real ship-it version needs.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)